

INTERVALNETS WHITE PAPER

****Repository:**** intervalNets
****Date:**** 2026-03-24

Executive Summary

intervalNets provides interval arithmetic and interval-aware neural-network evaluation with outward rounding. The repository combines a compact mathematical core ('interval.py') with a PyTorch integration layer ('pytorch.py') that overloads model evaluation on interval inputs and adds certified bounds for Jacobians, (L^p) norms, and Sobolev-style norms. This document focuses on those two files.

Repository Overview

At a high level, the codebase separates concerns:

- 'src/intervalnets/interval.py': scalar and nested-tuple interval representation, shape-aware operations, and outward-rounded arithmetic primitives.
- 'src/intervalnets/pytorch.py': PyTorch-facing interval tensor wrapper, layer-wise interval forward propagation, interval Jacobian propagation, and adaptive box refinement for norm enclosures.

The architectural pattern is: ****core numeric enclosure logic first****, then ****framework adaptation****.

Section: 'interval.py'

1) Purpose

'interval.py' defines an immutable 'Interval' datatype for closed intervals with outward rounding so results safely enclose floating-point roundoff. It supports scalar intervals and recursively nested tuple-shaped data for vector/tensor-like structures.

2) Structured Code Breakdown

Top-level functions

- '_is_sequence(value)': identifies list/tuple containers.
- '_to_data(value)': converts nested data into canonical 'float | tuple[...]' representation.
- '_map_unary(value, op)': recursively applies unary operators over nested tuple leaves.
- '_map_binary(left, right, op)': recursively applies binary operators with shape checks.
- 'outward_lower(value)', 'outward_upper(value)': outward rounding wrappers using 'nextafter' toward $-\infty$ and $+\infty$.
- '_validate_bounds(lower, upper)': validates shape compatibility and $(\ell \leq u)$ constraints.
- '_contains(lower, upper, value)': recursive membership test.
- '_shape(value)': computes recursive tuple shape.
- '_neg(value)': recursive negation.

Classes and methods

- 'Interval' (frozen dataclass)
- '__post_init__': canonicalizes and validates bounds.
- 'point(value)': degenerate interval with outward expansion.
- 'from_bounds(lower, upper)': constructs from explicit bounds with outward adjustment.
- 'shape': returns tuple shape.
- 'contains(value)': enclosure membership.
- 'as_tuple()': returns '(lower, upper)'.
- arithmetic: '__add__', '__sub__', '__neg__', '__mul__', '__truediv__', plus reflected variants.
- '__repr__': debug representation.

Important helper utilities

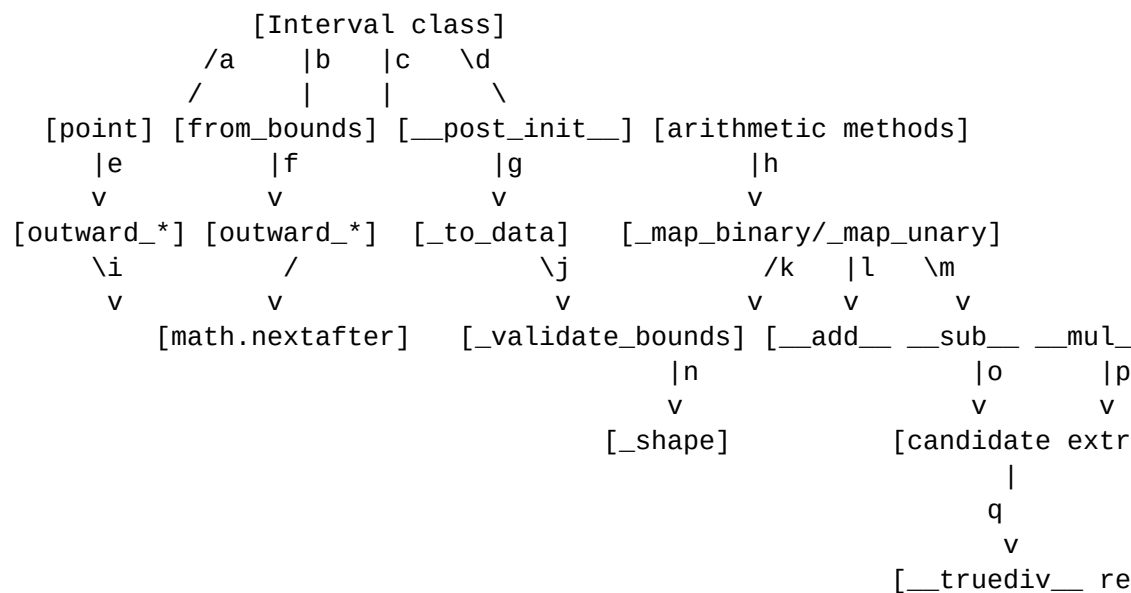
- Recursive mappers ('_map_unary', '_map_binary') are central abstraction points: most internal methods delegate leaf-level arithmetic to these functions.
- '_validate_bounds' and '_contains' encode structural consistency and safety predicates.

Important imports and dependencies

- Standard library only: 'dataclasses', 'math.inf', 'math.nextafter', typing helpers.
- No external numeric dependencies in this file.

3) Dependency / Mindmap Diagram

[diagram block]



l]

4) Arrow Legend

- (a) 'Interval.point' is a class entry-point used to create degenerate intervals.
- (b) 'Interval.from_bounds' is a class entry-point for explicit lower/upper inputs.
- (c) '__post_init__' normalizes constructor inputs immediately after dataclass creation.
- (d) Arithmetic dunder methods are methods on 'Interval'.
- (e) 'point' uses outward rounding utilities to widen exact points safely.
- (f) 'from_bounds' applies outward rounding to user-provided bounds.
- (g) '__post_init__' calls '_to_data' to canonicalize nested numeric types.
- (h) Arithmetic methods delegate leaf operations to recursive mapping helpers.
- (i) 'outward_lower' and 'outward_upper' both depend on 'nextafter' directionality.
- (j) '__post_init__' validates constraints through '_validate_bounds'.

- (k) `'__add__'` computes outward-rounded lower/upper sums via `'_map_binary'`.
- (l) `'__sub__'` pairs lower with upper (and vice versa) for worst-case subtraction.
- (m) `'__mul__'` either does elementwise tuple multiplication or scalar endpoint extrema.
- (n) shape queries (`'shape'` property) route through `'_shape'` recursion.
- (o) scalar multiplication inspects four endpoint products to bound extrema.
- (p) division is implemented as multiplication by an outward-rounded reciprocal interval.
- (q) reciprocal flow enforces zero-exclusion before inversion.

5) Mathematical Interpretation

- A closed interval is represented as $[\ell, u]$ with $\ell \leq u$.
- Outward rounding ensures computed interval I_{fp} encloses real arithmetic result $I_{\mathbb{R}}$:

$$[\ell, u] \subseteq I_{\text{fp}}$$
- Addition/subtraction follow endpoint rules:

$$[a, b] + [c, d] = [a+c, b+d]$$

$$[a, b] - [c, d] = [a-d, b-c]$$
then each endpoint is rounded outward.
- Scalar multiplication uses endpoint products:

$$[a, b] \cdot [c, d] = [\min(ac, ad, bc, bd), \max(ac, ad, bc, bd)]$$
- Division is transformed to multiplication by reciprocal interval when $0 \notin [c, d]$:

$$[a, b] / [c, d] = [a, b] \cdot [1/d, 1/c]$$

6) Implementation Notes

- Nested tuple support is recursive and shape-strict; mixed tuple/scalar paths are rejected
- Vector/tensor multiplication in this file is elementwise (same-shape tuple recursion), not matrix multiplication.
- Vector interval division is intentionally unimplemented (`'NotImplementedError'`).
- The dataclass is frozen, so methods return new intervals instead of mutating state.

Section: `'pytorch.py'`

1) Purpose

`'pytorch.py'` bridges the pure interval core to PyTorch modules. It introduces `'IntervalTensor'` interval forward propagation for supported layers, Jacobian interval bounds, and adaptive integration for L^p and Sobolev norms. It monkey-patches `'nn.Module'` to expose `'model.eval(interval)'`, `'model.lpnorm(...)'`, `'model.eval_jacobian(...)'`, and `'model.sobolev_norm(...)'`.

2) Structured Code Breakdown

Top-level functions

Core orchestration and helpers include:

- Environment and conversion: `'_require_torch'`, `'IntervalTensor.point'`, `'IntervalTensor.from_bounds'`, `'IntervalTensor.to_torch'`.
- Layer forward helpers: `'_linear_forward'`, `'_relu_forward'`, `'_sigmoid_forward'`, `'_tanh_forward'`, `'_softplus_forward'`, `'_leaky_relu_forward'`, `'_softmax_forward'`.
- Composite helpers: `'_interval_add'`, `'_interval_cat'`, `'_logsumexp'`, `'_softmax_component_bounds'`.
- Norm machinery: `'_box_volume'`, `'_lp_pointwise_power_bounds'`, `'_split_box'`, `'_lpnorm_bounds'`.
- Jacobian machinery: `'_identity_jacobian'`, `'_matrix_multiply'`, `'_jacobian_for_layer'`, `'_eval_jacobian_bounds'`.
- Sobolev machinery: `'_sobolev_pointwise_power_bounds'`, `'_sobolev_norm_bounds'`.
- Public dispatch/patch: `'interval_forward'`, `'enable_interval_eval'`.

Classes and methods

- `'IntervalTensor(Interval)'`
 - class methods converting Python/Torch inputs to interval data.
 - `'to_torch'` converts interval endpoints back to torch tensors.
- `'IntervalAdd(nn.Module)'`
 - forwards one input through two branches and adds outputs.
- `'IntervalCat(nn.Module)'`
 - forwards one input through several branches and concatenates outputs.

Important helper utilities

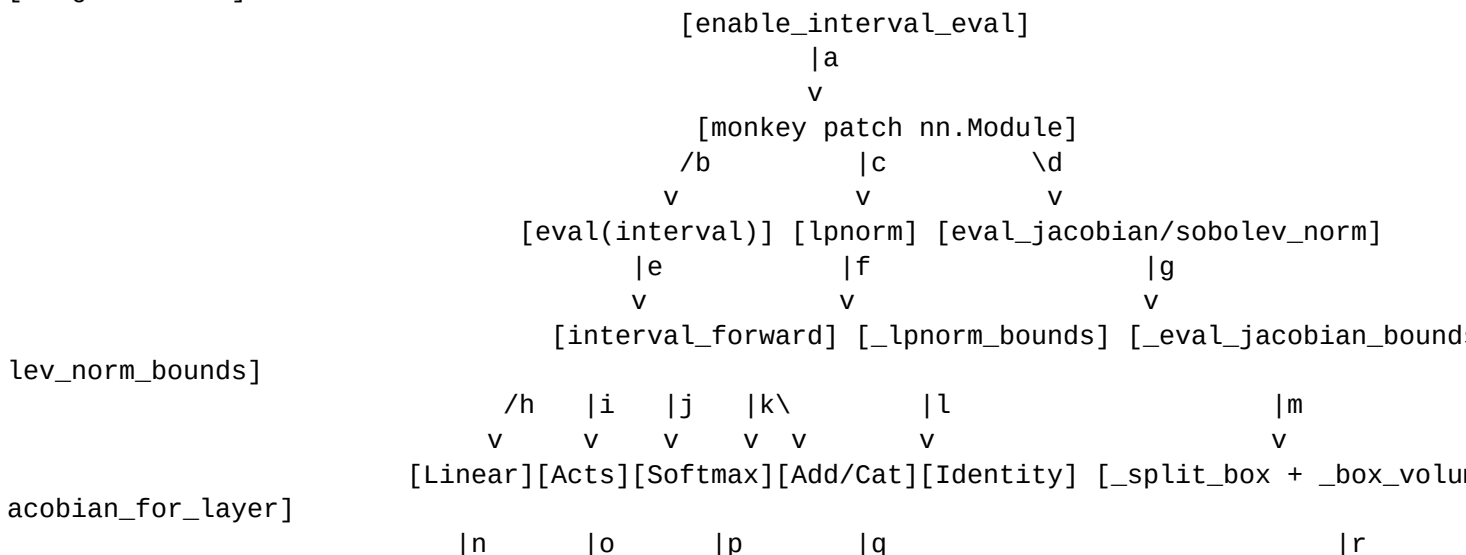
- `'_scalar_interval_from_weight'`: robustly encloses scalar coefficients, including explicit `'torch.nextafter'` for low-precision dtypes.
- `'_apply_monotone_bounds'`: endpoint-only propagation for monotone activations.
- `'_interval_abs_bounds'` and `'_interval_pow_scalar'`: scalar interval transformations used in integral bounds.
- `'_split_box'`: adaptive refinement by bisecting widest coordinate.

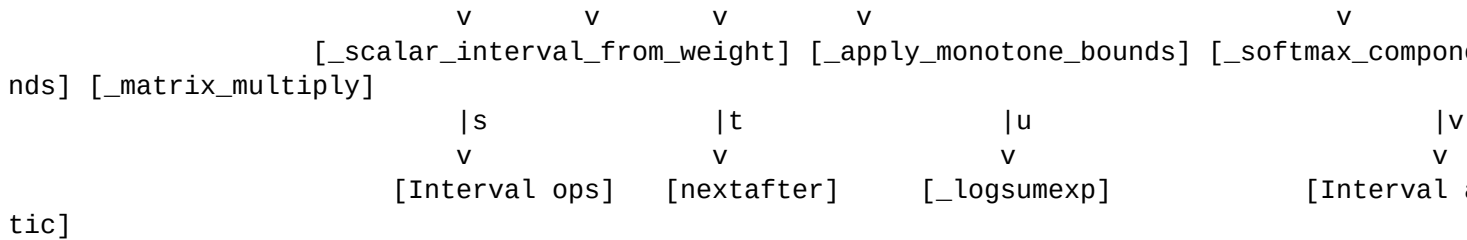
Important imports and dependencies

- Core dependency: `'torch'` and `'torch.nn'`.
- Reuses `'Interval'` from `'interval.py'`.
- Uses `'math'` functions (`'exp'`, `'log'`, `'tanh'`, `'nextafter'`, `'isfinite'`) for stable scalar computations.

3) Dependency / Mindmap Diagram

[diagram block]





4) Arrow Legend

- (a) 'enable_interval_eval' is the single entry for activating interval behavior.
- (b) Patched 'eval(interval)' routes interval input to interval forward propagation.
- (c) Patched 'lpnorm' routes to adaptive integral enclosure.
- (d) Patched Jacobian and Sobolev APIs route to derivative-aware enclosure routines.
- (e) 'eval(interval)' invokes 'interval_forward' dispatch by module type.
- (f) 'lpnorm' invokes '_lpnorm_bounds'.
- (g) Jacobian/Sobolev patched methods invoke '_eval_jacobian_bounds' / '_sobolev_norm_bounds'.
- (h) 'interval_forward' delegates linear layers to '_linear_forward'.
- (i) 'interval_forward' delegates monotone activations to dedicated helpers.
- (j) 'interval_forward' delegates Softmax to specialized bound logic.
- (k) branch combinators ('IntervalAdd', 'IntervalCat') route to structural interval helpers.
- (l) '_lpnorm_bounds' repeatedly uses '_split_box' and '_box_volume' for adaptive refinement.
- (m) Jacobian/Sobolev flows rely on '_jacobian_for_layer' (and then aggregation).
- (n) '_linear_forward' uses '_scalar_interval_from_weight' per coefficient/bias term.
- (o) activation helpers share '_apply_monotone_bounds' when monotonicity applies.
- (p) Softmax helper computes component extrema via '_softmax_component_bounds'.
- (q) branch combinator helpers merge interval outputs structurally.
- (r) Jacobian composition across layers is done by '_matrix_multiply'.
- (s) scalar weight enclosure multiplies by input intervals using 'Interval' arithmetic.
- (t) monotone endpoint propagation uses outward rounding via 'nextafter'.
- (u) Softmax extrema use '_logsumexp' for numerical stability.
- (v) Jacobian matrix products are interval-valued sums/products.

5) Mathematical Interpretation

- Monotone activation propagation uses

$$\begin{aligned} & \backslash[\\ & f([\ell, u]) = [f(\ell), f(u)] \\ & \backslash] \end{aligned}$$
 for increasing f , followed by outward rounding.
- Linear layer propagation encloses

$$\begin{aligned} & \backslash[\\ & y_i = \sum_j w_{ij} x_j + b_i \\ & \backslash] \end{aligned}$$
 by replacing scalars and inputs with intervals and applying interval arithmetic.
- Softmax component bounds compute exact box extrema by adversarial endpoint assignment per component i :

$$\begin{aligned} & \backslash[\\ & \sigma_i(x) = \frac{e^{x_i}}{\sum_j e^{x_j}}. \\ & \backslash] \end{aligned}$$
- L^p -norm enclosure integrates interval bounds of $|f(x)|^p$ over a box domain and applies

$$\begin{aligned} & \backslash[\\ & \|f\|_{L^p} = \left(\int |f(x)|^p dx \right)^{1/p}. \\ & \backslash] \end{aligned}$$
- Sobolev-style enclosure similarly accumulates powers of function outputs and Jacobian entries.

before integration.

6) Implementation Notes

- Most interval network operations currently assume flat 1D vectors; unsupported shapes raise explicit `'NotImplementedError'`.
- Optional PyTorch dependency is guarded (`'try/except ImportError'`) and validated via `'_require_torch'`.
- Monkey patching is global (`'nn.Module'`), one-way for process lifetime, and guarded by `'_PATCHED'`.
- Adaptive integration chooses the box with largest indicator $'(\text{integrand width}) * (\text{box volume})'$ for bisection.

Conclusion

`'interval.py'` and `'pytorch.py'` together form a coherent two-layer design: mathematically conservative interval primitives, then framework-level propagation and integration algorithms built on top. The implementation is intentionally explicit about supported cases, error behavior, and outward-rounding guarantees, making it suitable for certified-enclosure workflows in neural-network analysis.